

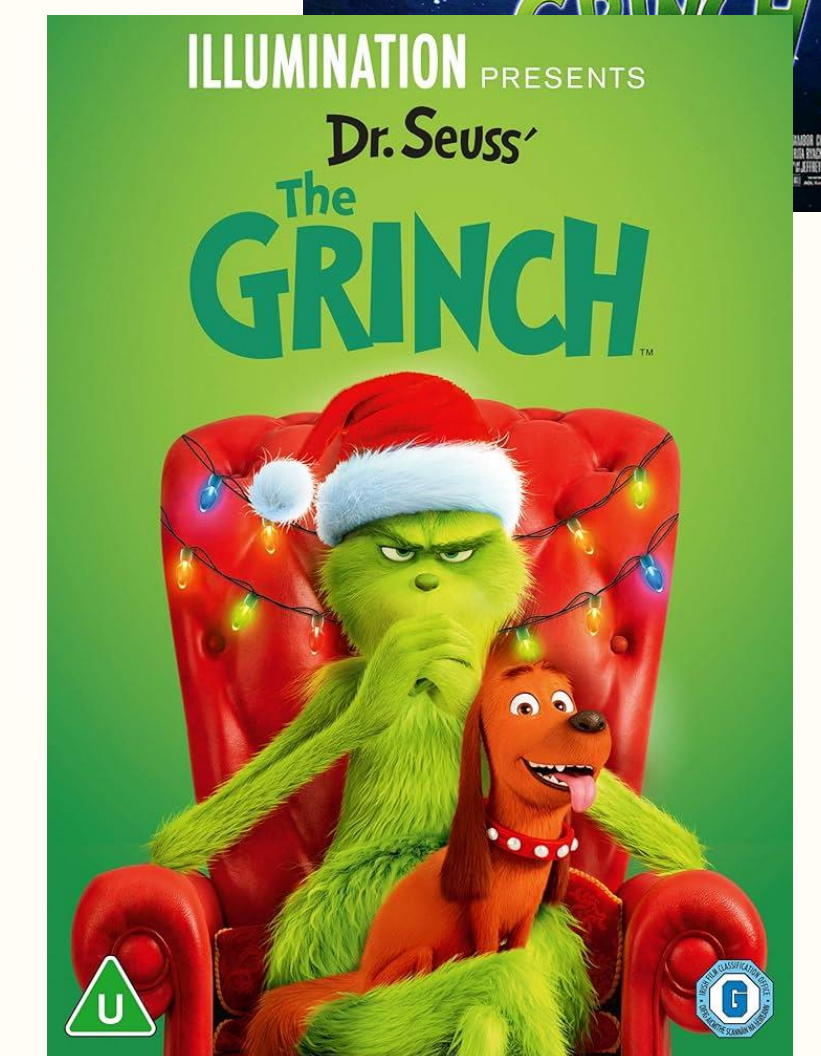
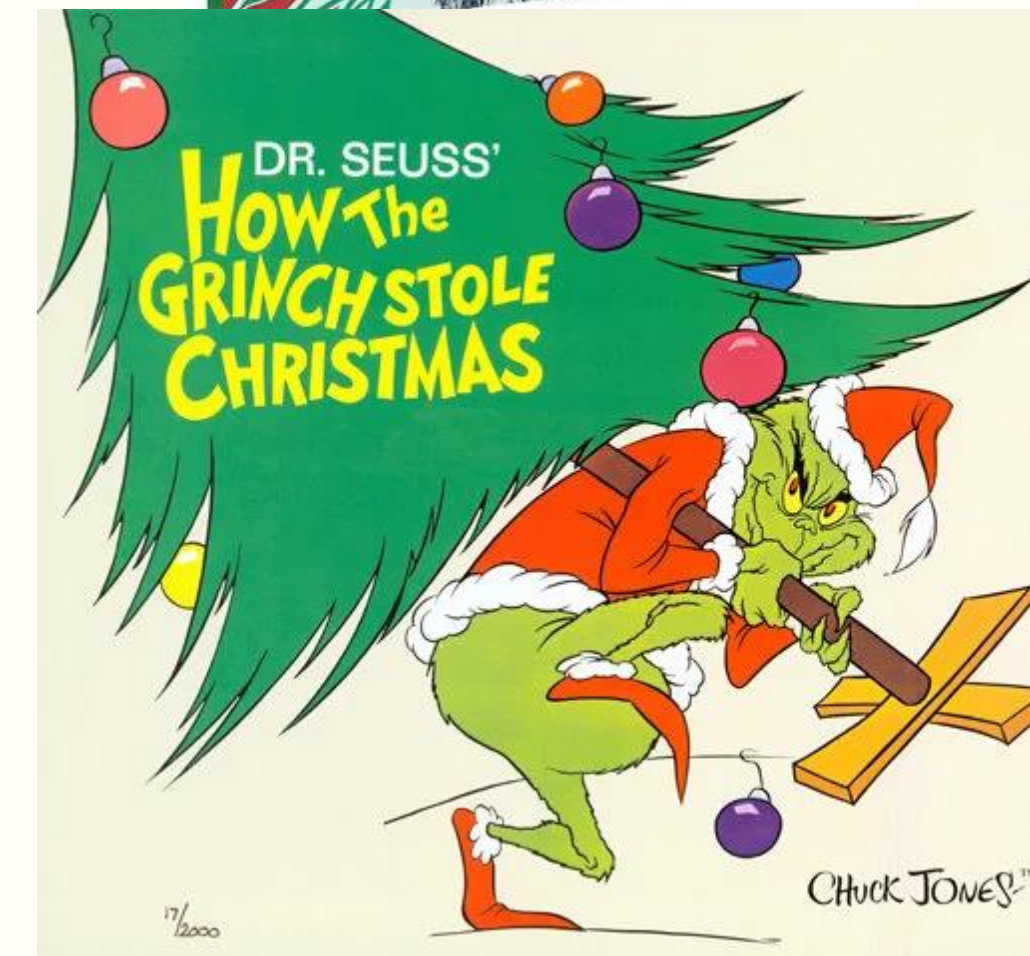
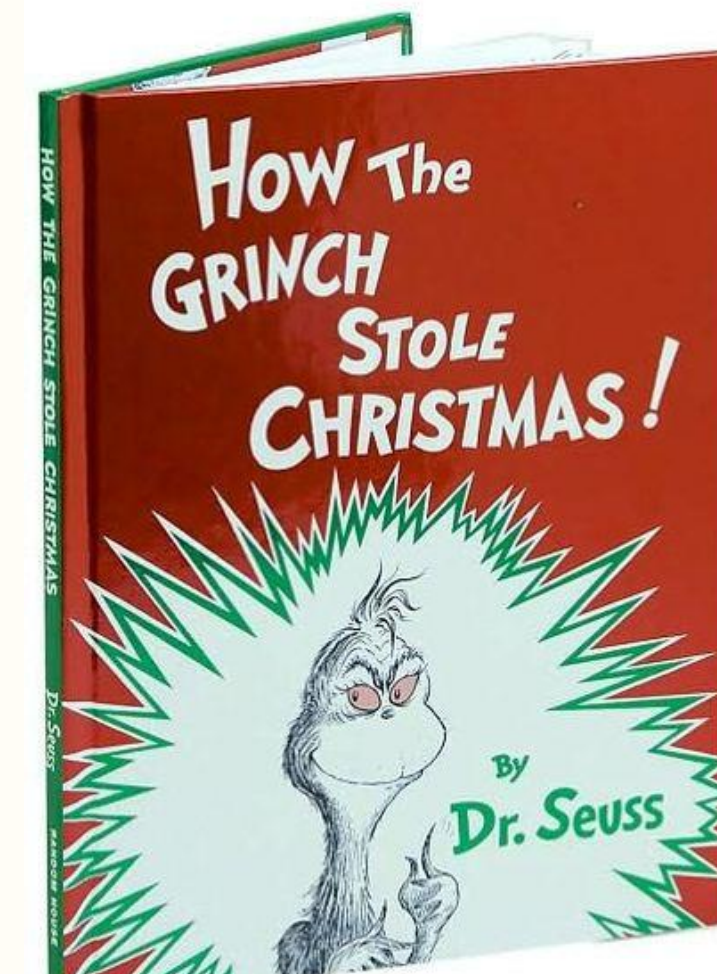
Lecture 24: Machine Learning Continued

Heather Mattie, PhD

December 2, 2025

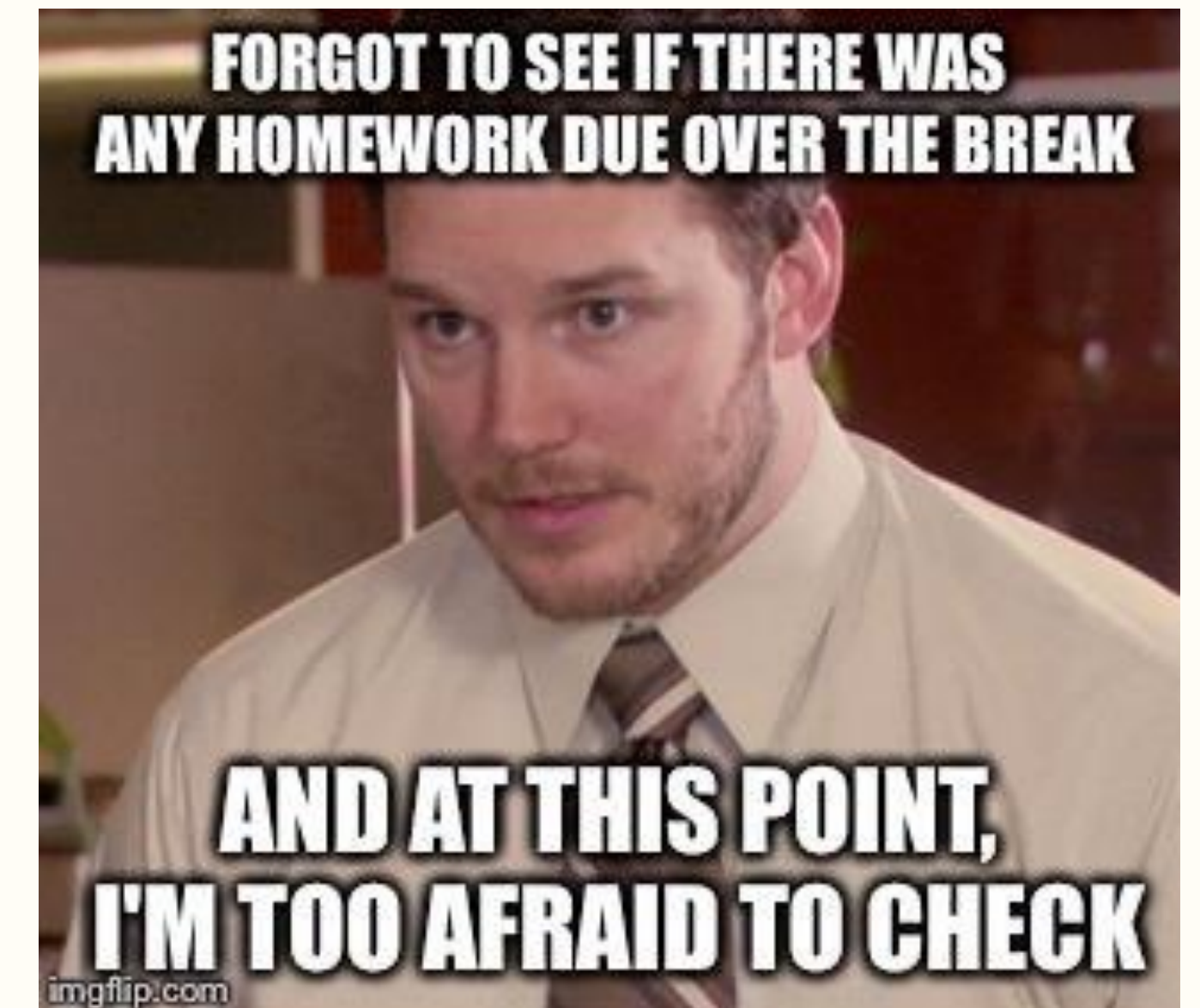
Recipe of the Day!

Grinch Cookies



Announcements

- Lab this week on machine learning
- Homework #4 due Friday, December 5th
- Homework #5 due Friday, December 12th



Machine Learning

Cross validation

Heather Mattie, PhD

Logistic regression vs Naïve Bayes

Algorithm	Overall accuracy	Sensitivity	Specificity	F ₁ score	AUROC
Logistic regression	0.706	0.2174	0.9556	0.333	0.794
Naïve Bayes	0.706	0.2174	0.9556	0.333	0.768

Note: identical confusion matrices reflect identical decisions at **one cutoff** (0.5), while different AUROC values reflect differences in the underlying predicted probabilities across **all cutoffs**.

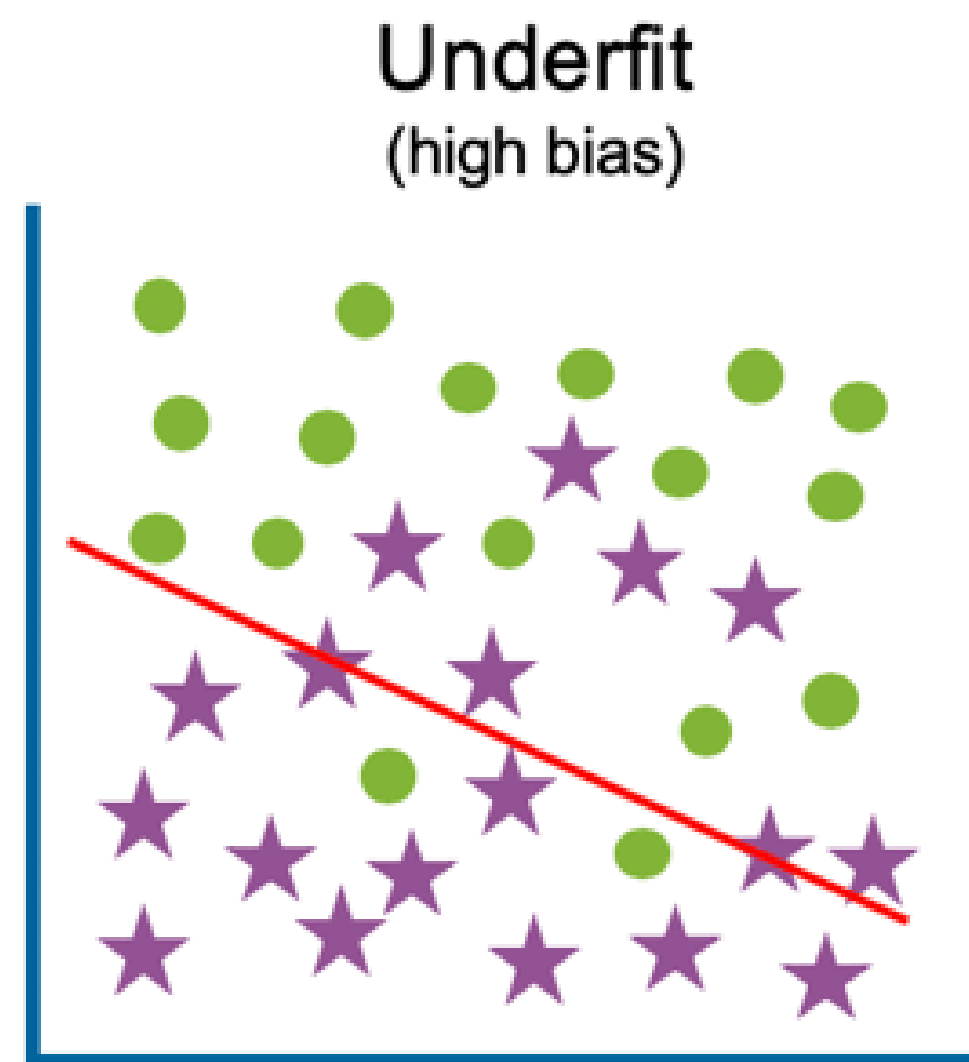
- Looking at AUROC, the logistic regression model performs slightly better

Overfitting

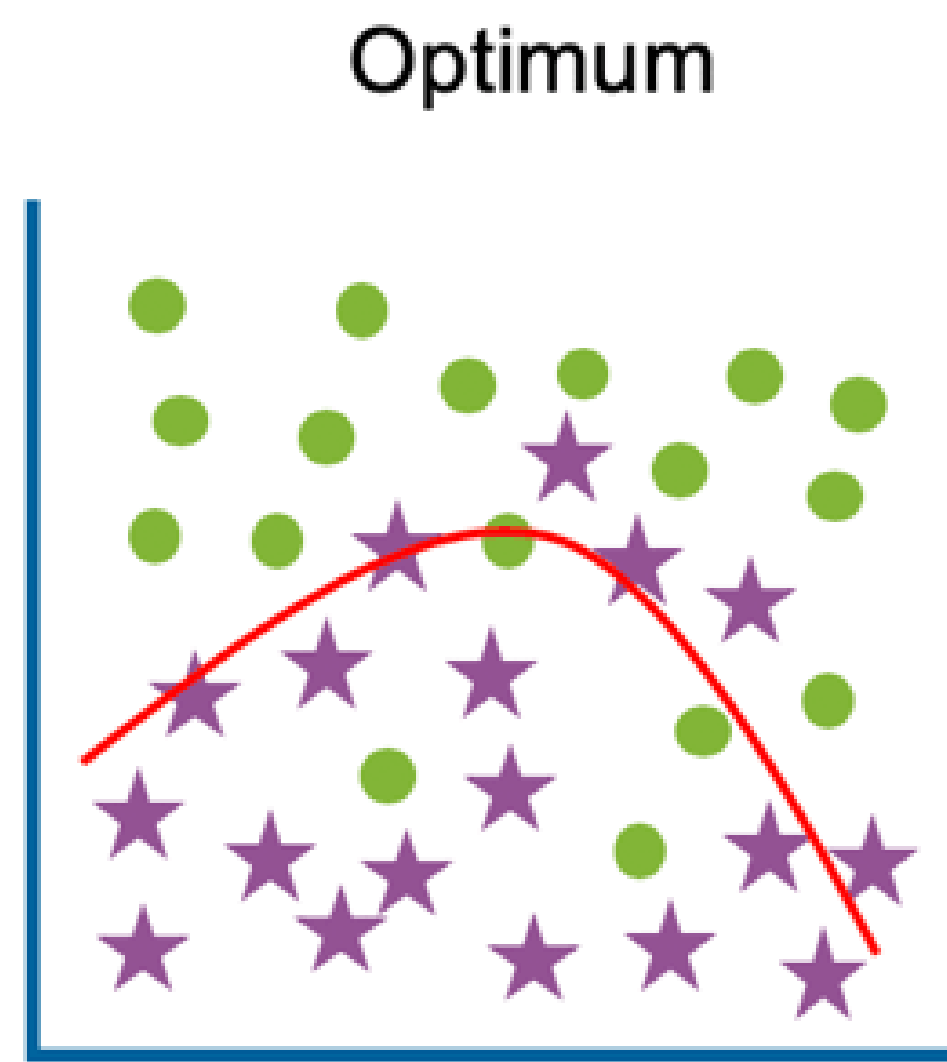
Overfitting is when a machine learning model learns the training data too well, including its noise and irrelevant details, leading to high accuracy on the training data but poor performance on new, unseen data.

- The model has become too complex and has memorized the training set rather than learning the general patterns.
- Leads to poor generalization, making predictions unreliable in real-world settings
- More likely with small datasets or highly flexible algorithms (e.g., deep trees, neural nets)
- Can be detected by comparing training vs. validation/test performance
- Prevented through cross-validation, regularization, simpler models, or more data

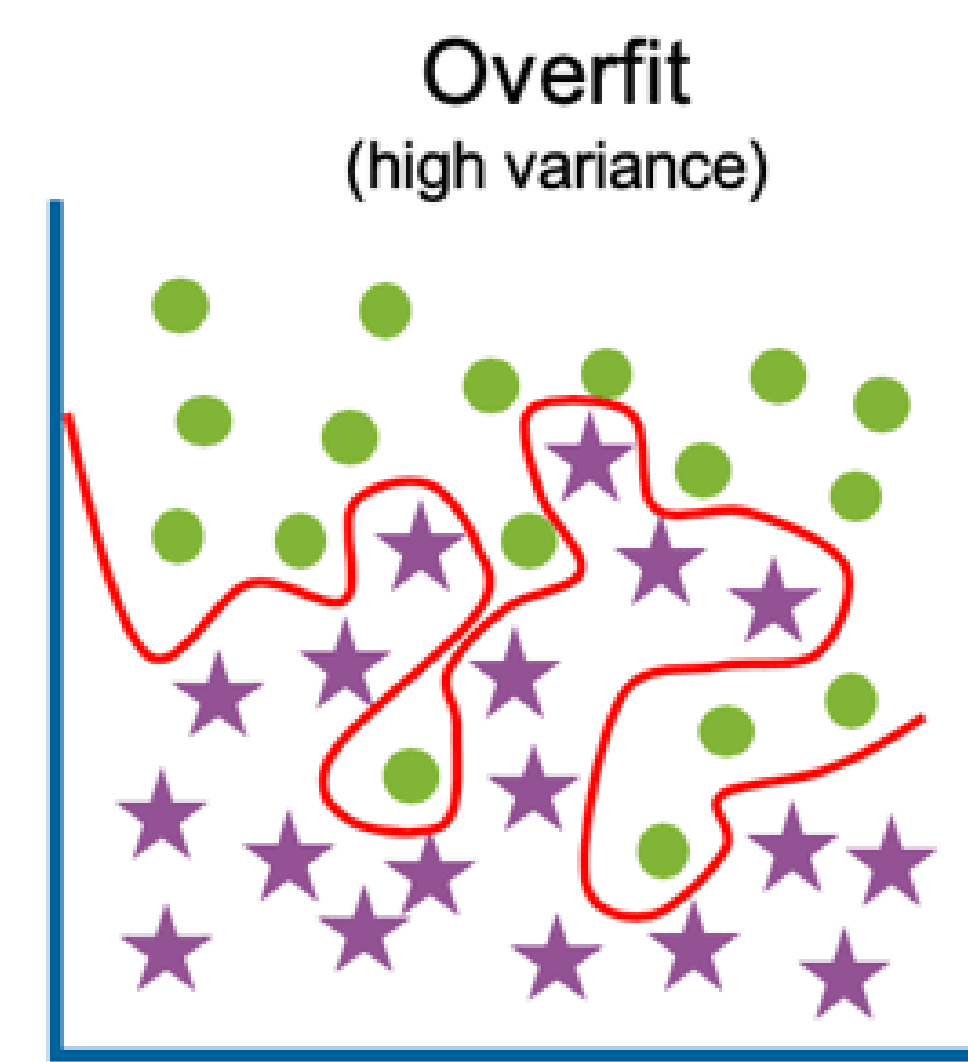
Overfitting



High training error
High test error



Low training error
Low test error

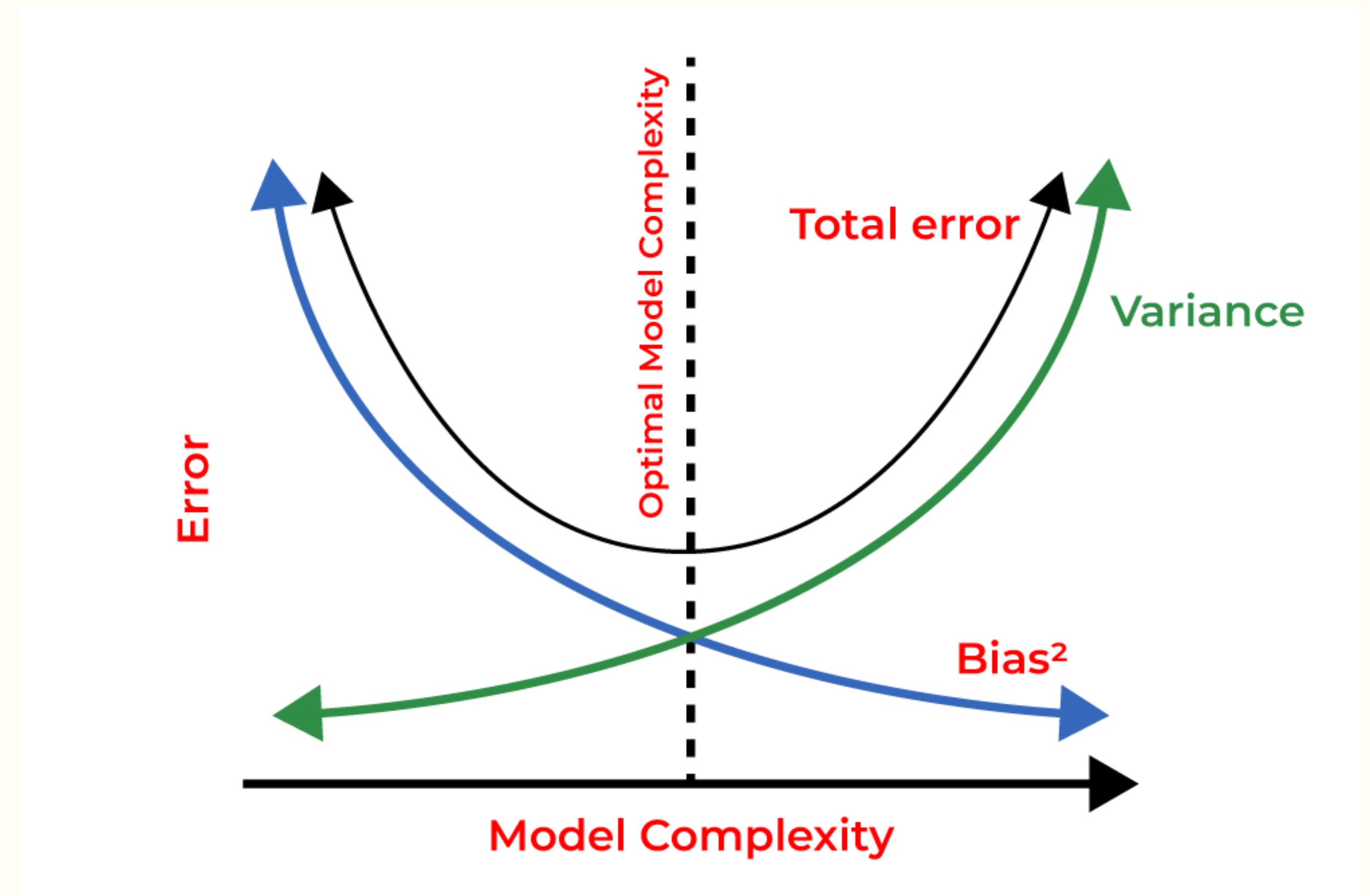


Low training error
High test error

Bias and variance

ML bias: the error that occurs when a model is **too simple** to capture the true patterns in the data.

ML variance: how much a model's predictions change when it's trained on different data.



Cross-validation

Cross-validation is a method for estimating how well a machine learning model will generalize to new, unseen data.

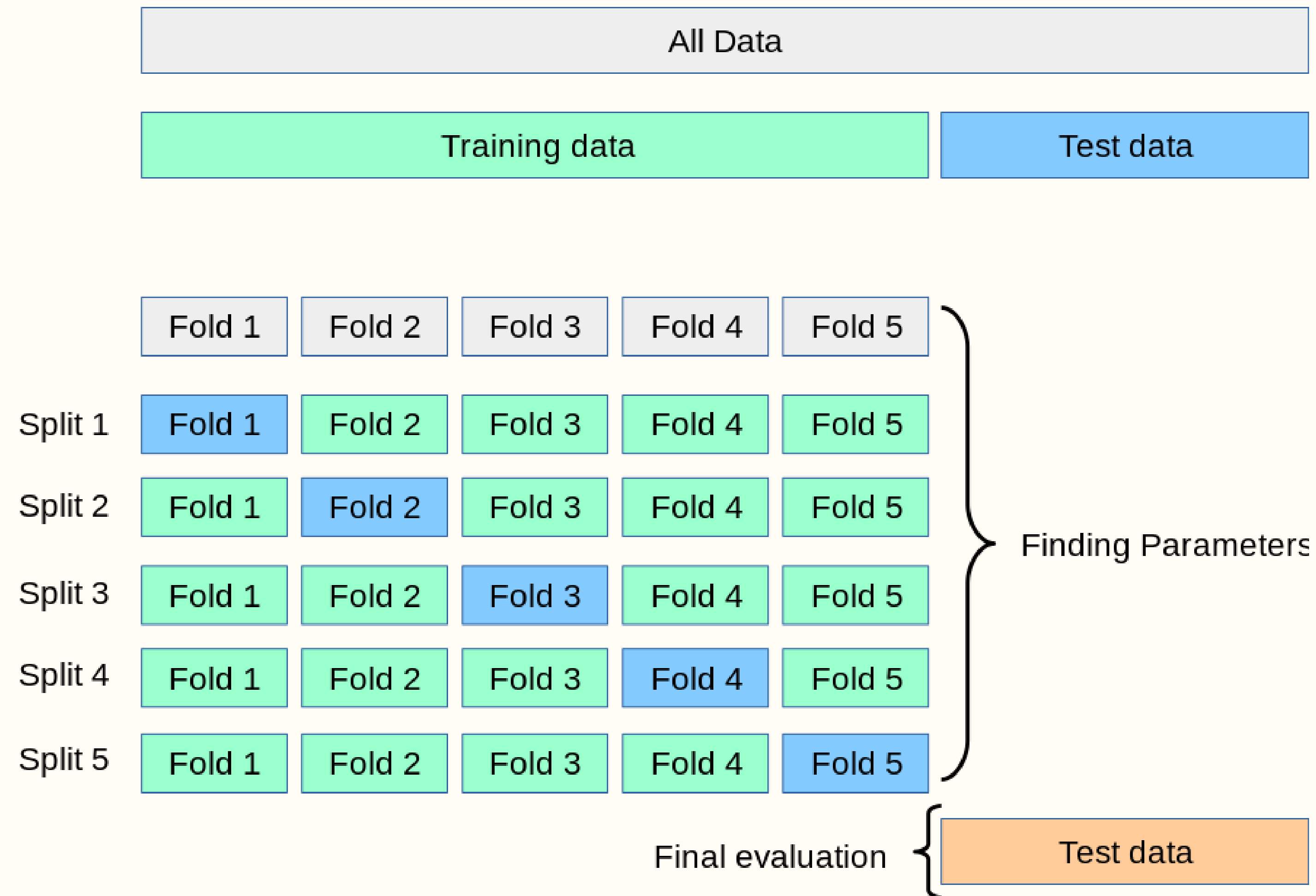
- Helps detect **overfitting** by testing the model on data it wasn't trained on
- Uses repeated train/test splits to get a more stable estimate of performance
- **Useful when the dataset is small**, since all data contribute to both training and evaluation
- Produces more reliable performance metrics than a single train/test split

k-fold cross-validation

With **k-fold cross-validation**, the dataset is divided into **k equal-sized folds (partitions)**. The most common values for k are 5 and 10.

- The model is trained k times, each time using a different fold as the test set
- Performance is averaged across all k runs, giving a stable estimate
- Ensures every observation is used for both training and testing
- Reduces variability in model performance estimates, especially on small datasets

k-fold cross-validation



Case study: melanoma

```
library(tidyverse)
library(boot)
data(melanoma)

melanoma <- melanoma %>%
  filter(status %in% c(1, 2)) %>%
  mutate(status = as.factor(ifelse(status == 1, "Dead", "Alive")),
         y = as.factor(ifelse(status == "Dead", 1, 0)))
```

```
head(melanoma)
```

##	time	status	sex	age	year	thickness	ulcer	y
## 3	35	Alive	1	41	1977	1.34	0	0
## 5	185	Dead	1	52	1965	12.08	1	1
## 6	204	Dead	1	28	1971	4.84	1	1
## 7	210	Dead	1	77	1972	5.16	1	1
## 9	232	Dead	1	49	1968	12.88	1	1
## 10	279	Dead	0	68	1971	7.41	1	1

trainControl

The **trainControl** function from the **caret** package is used to control the computational nuances and resampling procedures for the **train** function.

- Allows users to specify how models should be trained and evaluated, particularly concerning cross-validation and other resampling methods.

```
melanoma <- melanoma %>%  
  filter(status %in% c(1, 2)) %>%  
  mutate(status = as.factor(ifelse(status == 1, "Dead", "Alive")),  
         status = factor(status,  
                        levels = c("Alive", "Dead"),  
                        labels = c("Negative", "Positive")),  
         y = as.factor(ifelse(status == "Dead", 1, 0)))
```

```
set.seed(123)  
  
cv10 <- trainControl(  
  method = "cv",  
  number = 10,  
  classProbs = TRUE,  
  summaryFunction = twoClassSummary  
)
```

Logistic regression

```
log_cv <- train(  
  status ~ thickness,  
  data = train_set,  
  method = "glm",  
  family = binomial,  
  trControl = cv10,  
  metric = "ROC"  
)
```

```
log_cv
```

```
## Generalized Linear Model  
##  
## 134 samples  
## 1 predictor  
## 2 classes: 'Alive', 'Dead'  
##  
## No pre-processing  
## Resampling: Cross-Validated (10 fold)  
## Summary of sample sizes: 120, 120, 121, 121, 121, 121, ...  
## Resampling results:  
##  
## ROC Sens Spec  
## 0.749 0.911 0.175
```

Logistic regression

```
p_hat_logit_cv <- predict(log_cv, test_set, type = "prob")[,2]

y_hat_logit_cv <- factor(ifelse(p_hat_logit_cv >= 0.5, "Dead", "Alive"),
                          levels=c("Alive","Dead"))

confusionMatrix(y_hat_logit_cv, test_set$status, positive = "Dead")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Alive Dead
##      Alive    43   18
##      Dead     2    5
##
##              Accuracy : 0.706
```

```
##           Sensitivity : 0.2174
##           Specificity : 0.9556
##           Pos Pred Value : 0.7143
##           Neg Pred Value : 0.7049
##           Prevalence : 0.3382
##           Detection Rate : 0.0735
##           Detection Prevalence : 0.1029
##           Balanced Accuracy : 0.5865
##
##           'Positive' Class : Dead
##
```

No change for this model compared to when k-fold cross-validation was not used.

Naïve Bayes

```
library(naivebayes)

nb_cv <- train(
  status ~ thickness,
  data = train_set,
  method = "naive_bayes",
  trControl = cv10,
  metric = "ROC"
)

nb_cv
```

```
## Naive Bayes
##
## 134 samples
## 1 predictor
## 2 classes: 'Alive', 'Dead'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 121, 121, 121, 120, 120, 121, ...
## Resampling results across tuning parameters:
##
## usekernel ROC Sens Spec
## FALSE 0.701 0.916 0.175
## TRUE 0.743 0.903 0.325
##
## Tuning parameter 'laplace' was held constant at a value of 0
## Tuning
## parameter 'adjust' was held constant at a value of 1
## ROC was used to select the optimal model using the largest value.
## The final values used for the model were laplace = 0, usekernel = TRUE
## and adjust = 1.
```

Naïve Bayes

```
p_hat_nb_cv <- predict(nb_cv, test_set, type = "prob")[,2]

y_hat_nb_cv <- factor(ifelse(p_hat_nb_cv >= 0.5, "Dead", "Alive"),
                      levels=c("Alive", "Dead"))

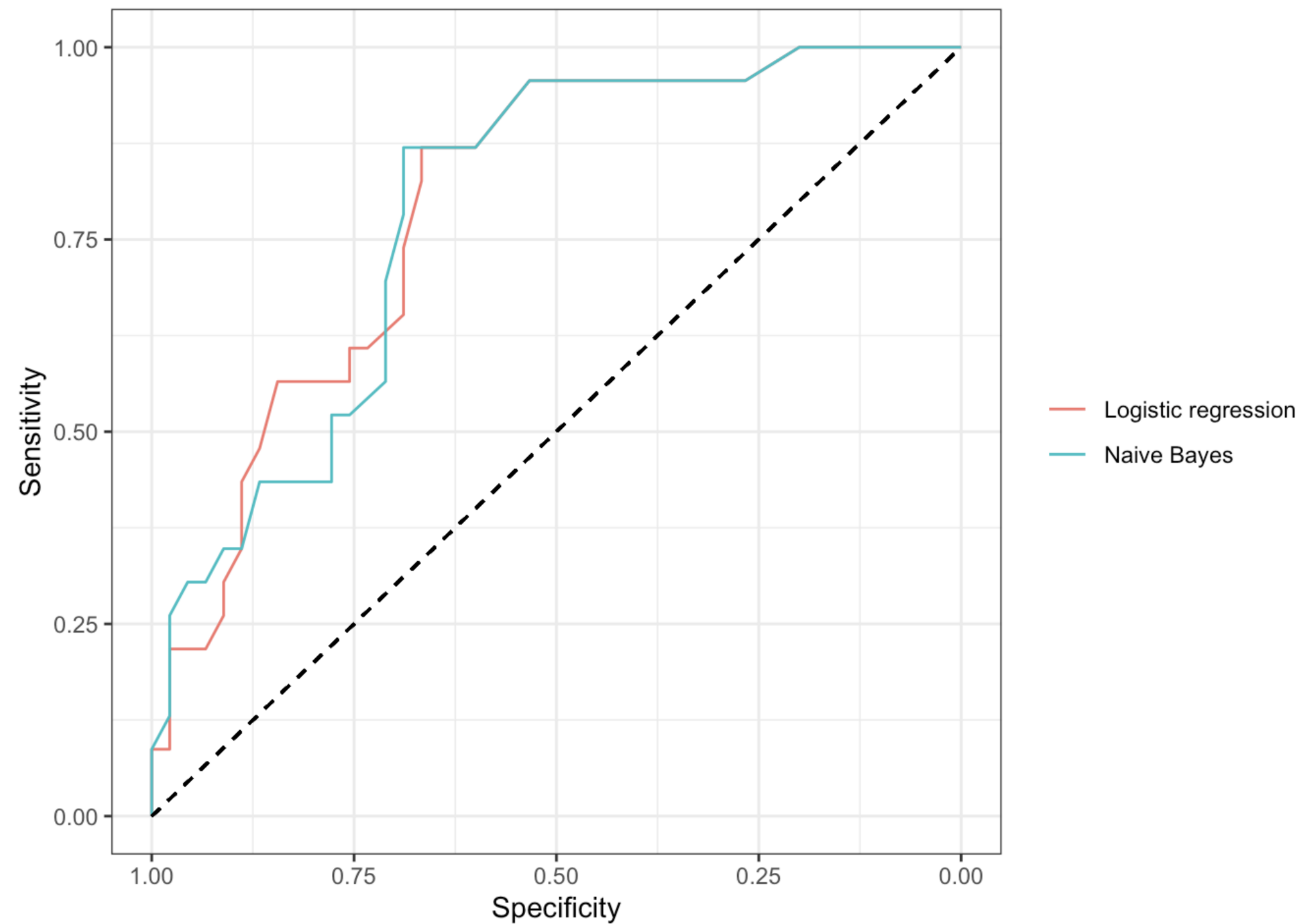
confusionMatrix(y_hat_nb_cv, test_set$status, positive = "Dead")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Alive Dead
##      Alive    40   15
##      Dead     5    8
##
##              Accuracy : 0.706
```

```
##              Sensitivity : 0.348
##              Specificity : 0.889
##              Pos Pred Value : 0.615
##              Neg Pred Value : 0.727
##              Prevalence : 0.338
##              Detection Rate : 0.118
##              Detection Prevalence : 0.191
##              Balanced Accuracy : 0.618
##
##              'Positive' Class : Dead
##
```

The overall accuracy did not change, but the sensitivity increased and the specificity decreased.

ROC and AUROC



```
auc(roc_logit_cv)
```

```
## Area under the curve: 0.794
```

```
auc(roc_nb_cv)
```

```
## Area under the curve: 0.788
```

Logistic regression vs Naïve Bayes

Algorithm	Overall accuracy	Sensitivity	Specificity	F_1 score	AUROC
Logistic regression	0.706	0.2174	0.9556	0.333	0.794
Naïve Bayes	0.706	0.348	0.889	0.444	0.788

Choosing the best model is a little more difficult now. The Naïve Bayes model has the same overall accuracy, but a higher F_1 score and sensitivity, but a lower specificity. Overall, both models perform very similarly.

Machine Learning

k-Nearest Neighbors (KNN)

Heather Mattie, PhD

k-Nearest Neighbors (KNN)

k-Nearest Neighbors (KNN) is a simple machine learning algorithm for classification and regression that works by finding the ***k*** most similar data points (neighbors) to a new data point and assigning it to the most frequent class among those neighbors.

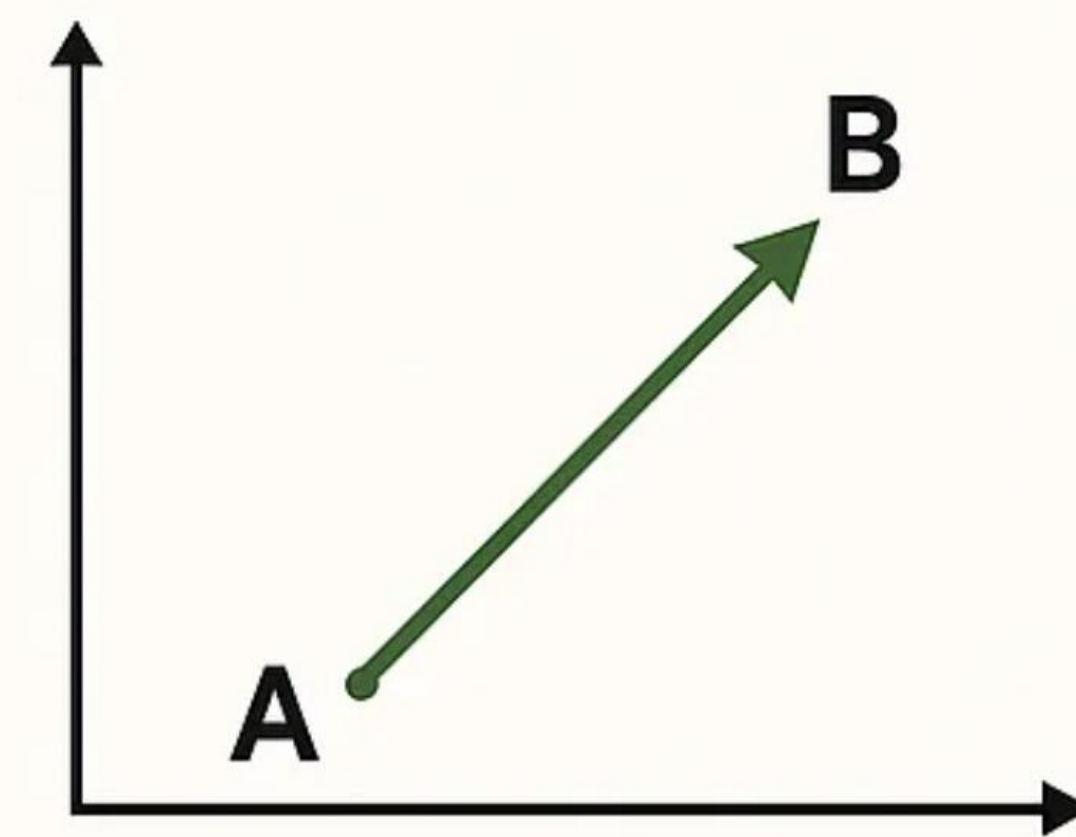
- Uses a distance metric (like Euclidean or Manhattan) to determine which data points are “nearest”
- Requires the user to define the value for ***k***



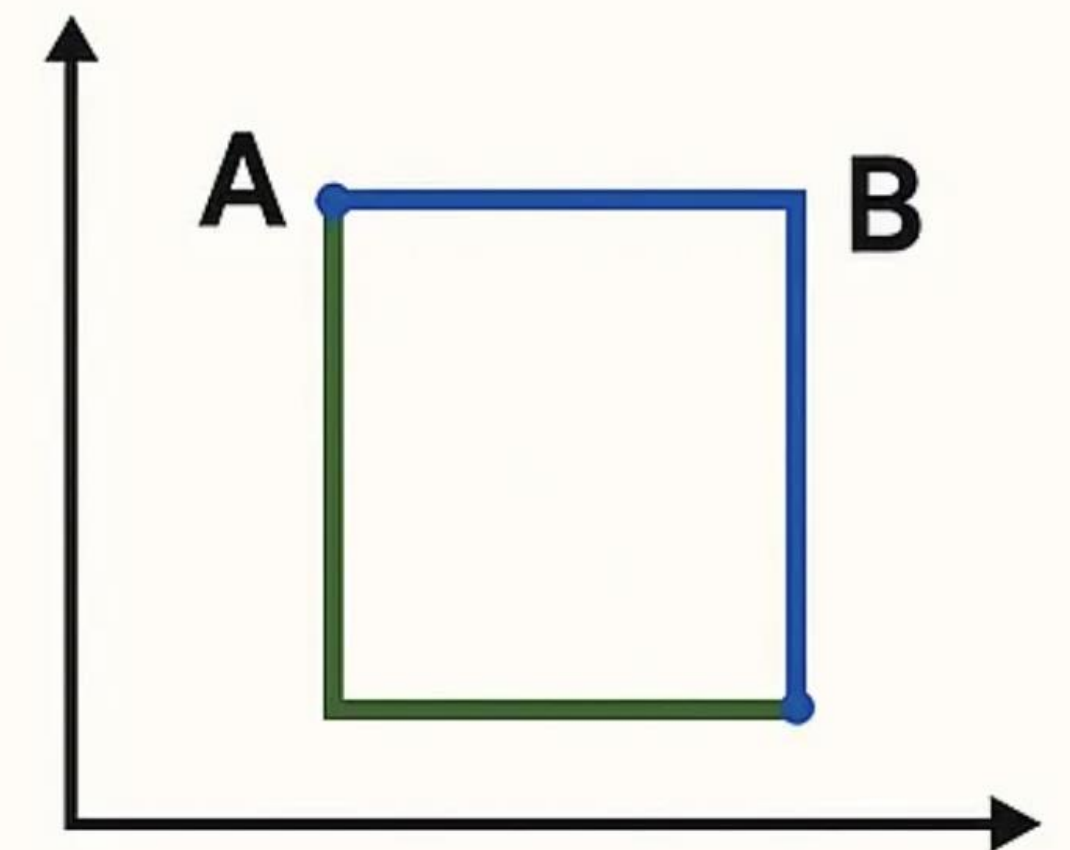
Distance measures

The most common distance measure used is Euclidean distance.

- Manhattan distance is used when the data is high-dimensional



Euclidean Distance



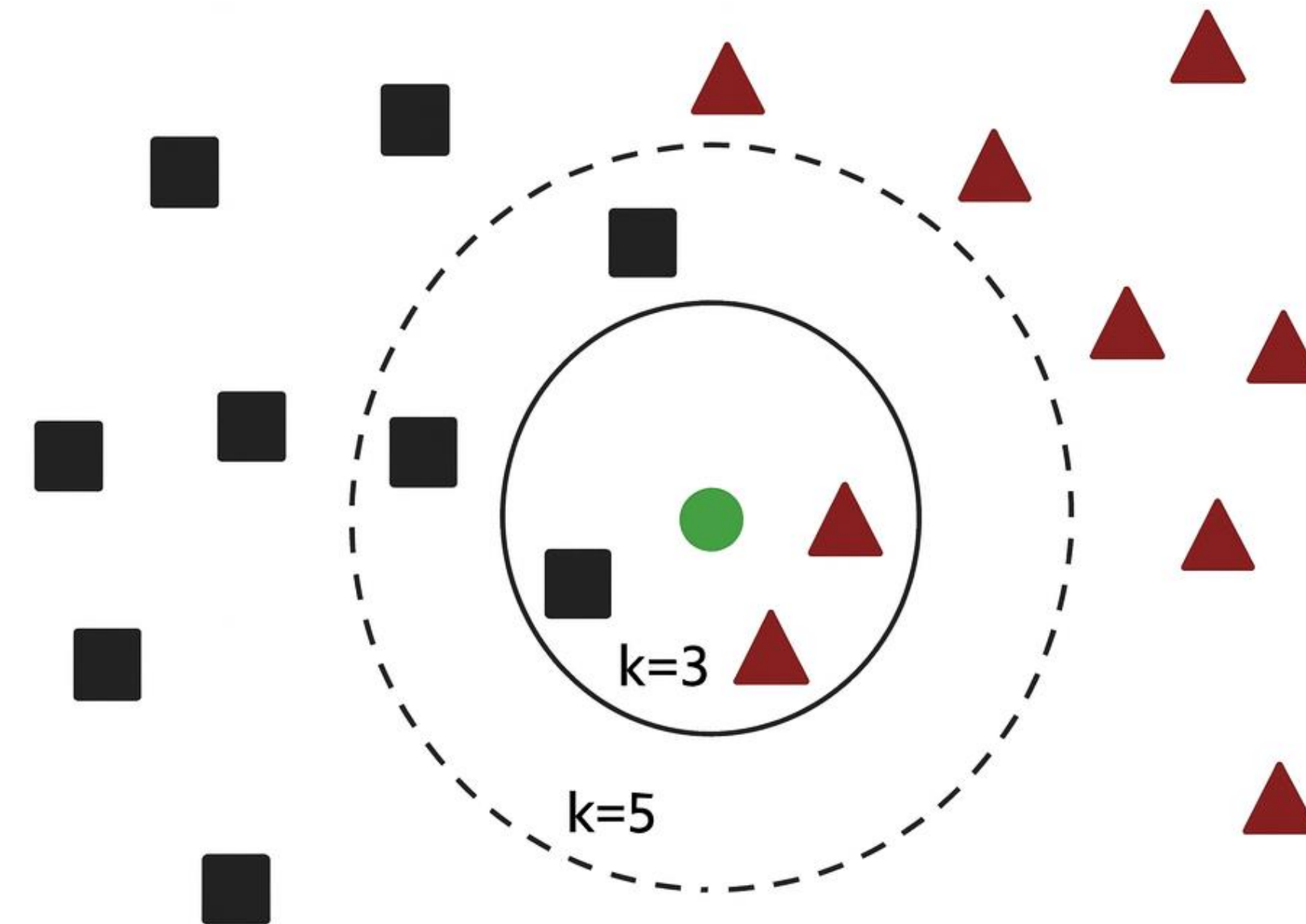
Manhattan Distance

Classification steps

1. To predict the class of a new observation, compute the distance between the new point and every observation in the training set.
2. Find the ***k*** closest (nearest) observations according to this distance.
3. Assign the new point the majority class among those neighbors.

k-Nearest Neighbors (KNN)

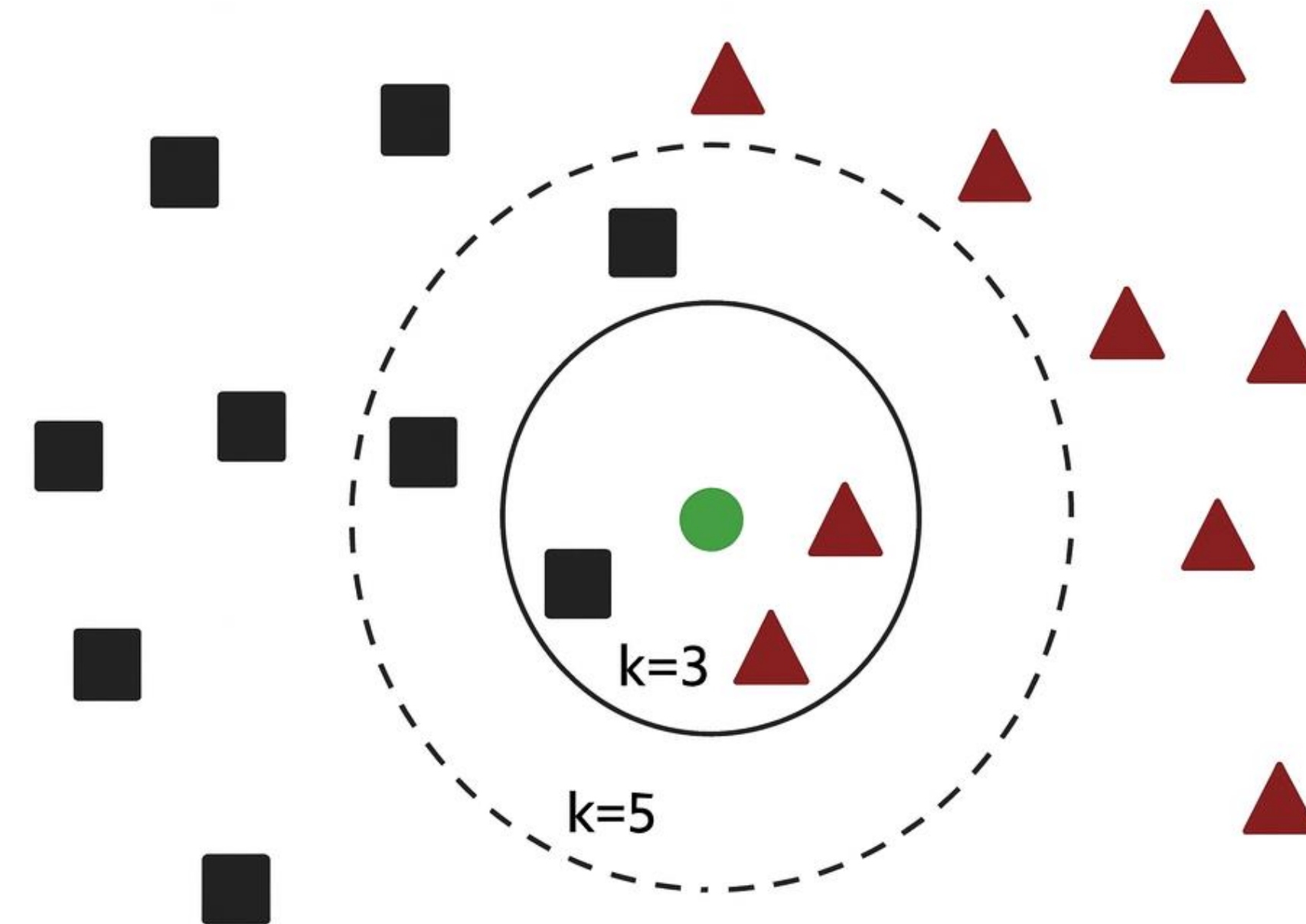
What class does the new data point belong to?



k-Nearest Neighbors (KNN)

- In this example, if $k = 3$, the new data point would be classified as a triangle
- If $k = 5$, the new data point would be classified as a square

What class does the new data point belong to?



k-Nearest Neighbors (KNN)

- KNN does not learn an explicit model with parameters, unlike logistic regression and Naive Bayes
- Predictions are based on local neighborhoods in the feature space (predictor space)
- The choice of distance metric and the scale of features matter a lot - small changes can result in large differences in predictions and prediction accuracy
- The tuning parameter ***k*** controls the flexibility:
 - Small *k* = more flexible, higher variance, more sensitive to noise, usually less bias
 - Large *k* = smoother decision boundary, higher bias
- Because KNN relies on distances, features with larger numerical ranges can dominate the distance calculation. Therefore, **standardizing** continuous predictors (e.g., centering and scaling) is necessary.

Cross-validation setup

The same cross-validation setup used with the logistic and Naïve Bayes models will be used here

- The **trainControl** function from the **caret** package is used to control the computational nuances and resampling procedures for the **train** function.
- Allows users to specify how models should be trained and evaluated, particularly concerning cross-validation and other resampling methods.

```
set.seed(123)

cv10 <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = twoClassSummary
)
```

Model training

- Need to define a list of values to try for ***k***
 - Odd numbers are better since classification relies on a majority vote of neighbors
- **method = "knn"** for KNN algorithm
- **trControl = cv10** to specify the cross-validation scheme
- **preProcess = c("center", "scale")** so that KNN uses standardized predictors
- **tuneGrid** to define which ***k*** values to try
- **metric = "ROC"** to have the best ***k*** chosen based on highest average AUROC

```
k_grid <- data.frame(k = seq(3, 25, by = 2))

knn_cv <- train(
  status ~ thickness,
  data = train_set,
  method = "knn",
  trControl = cv10,
  preProcess = c("center", "scale"),
  tuneGrid = k_grid,
  metric = "ROC"
)

knn_cv
```

The optimal *k*

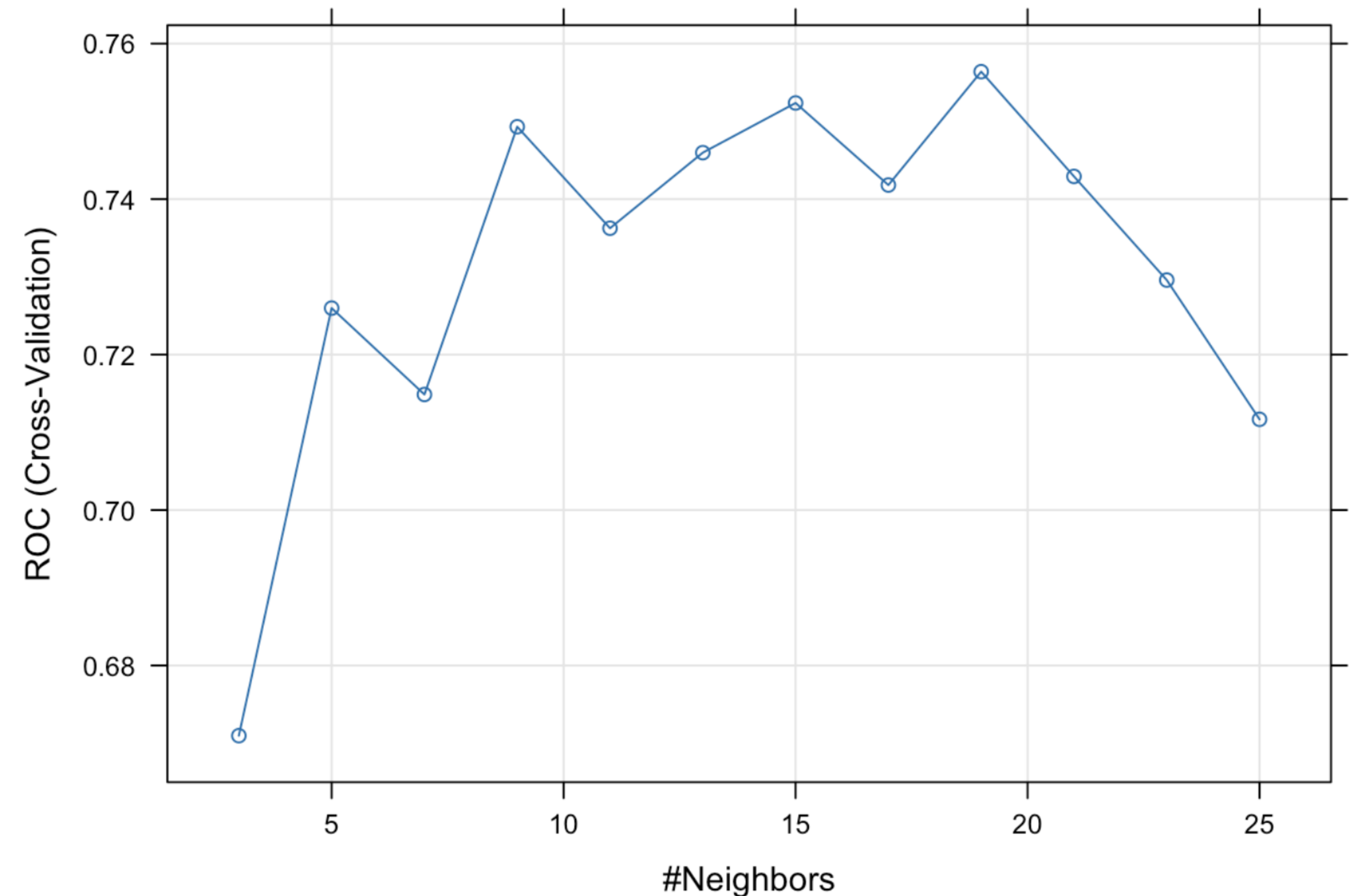
- The output of the train function contains the optimal value for *k* at the bottom
- The optimal value was chosen based on highest average AUROC value
- For each *k*, caret generates 10 accuracy estimates (because of 10-fold cross-validation), and then averages them

```
## k-Nearest Neighbors
##
## 134 samples
## 1 predictor
## 2 classes: 'Alive', 'Dead'
##
## Pre-processing: centered (1), scaled (1)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 120, 120, 121, 121, 121, 121, ...
## Resampling results across tuning parameters:
##
## k   ROC    Sens   Spec
## 3   0.671  0.864  0.425
## 5   0.726  0.853  0.350
## 7   0.715  0.852  0.425
## 9   0.749  0.874  0.325
## 11  0.736  0.863  0.325
## 13  0.746  0.864  0.300
## 15  0.752  0.884  0.225
## 17  0.742  0.894  0.275
## 19  0.756  0.883  0.250
## 21  0.743  0.861  0.250
## 23  0.730  0.850  0.350
## 25  0.712  0.839  0.300
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was k = 19.
```

The optimal k

- The `plot` function can be used to view the training set accuracy (AUROC) for each value of k
- It is common to see an increase in accuracy as k increases, but then the accuracy will plateau or start to decrease, indicating underfitting is happening

```
plot(knn_cv)
```



Test set predictions

- The `type = "prob"` argument in the `predict` function returns predicted probabilities for each class
- Including `[,2]` retains the predicted probabilities for the 1 class (`[,1]` is the predicted probabilities for the 0 class)

```
# Predictions on the test set
p_hat_knn_cv <- predict(knn_cv, newdata = test_set, type = "prob")[,2]

# Converting probabilities to 0 or 1
y_hat_knn_cv <- factor(ifelse(p_hat_knn_cv >= 0.5, "Dead", "Alive"),
                      levels = c("Alive", "Dead"))

# Confusion matrix
cm_knn <- confusionMatrix(y_hat_knn_cv, test_set$status, positive = "Dead")
cm_knn
```

Test set performance

- This model has the lowest overall accuracy
- High specificity, low sensitivity

```
## Confusion Matrix and Statistics
##
##               Reference
## Prediction Alive Dead
##      Alive    40   17
##      Dead     5    6
##
##               Accuracy : 0.676
##               95% CI : (0.552, 0.785)
##      No Information Rate : 0.662
##      P-Value [Acc > NIR] : 0.455
##
##               Kappa : 0.172
##
##      Mcnemar's Test P-Value : 0.019
##
##               Sensitivity : 0.2609
##               Specificity : 0.8889
```

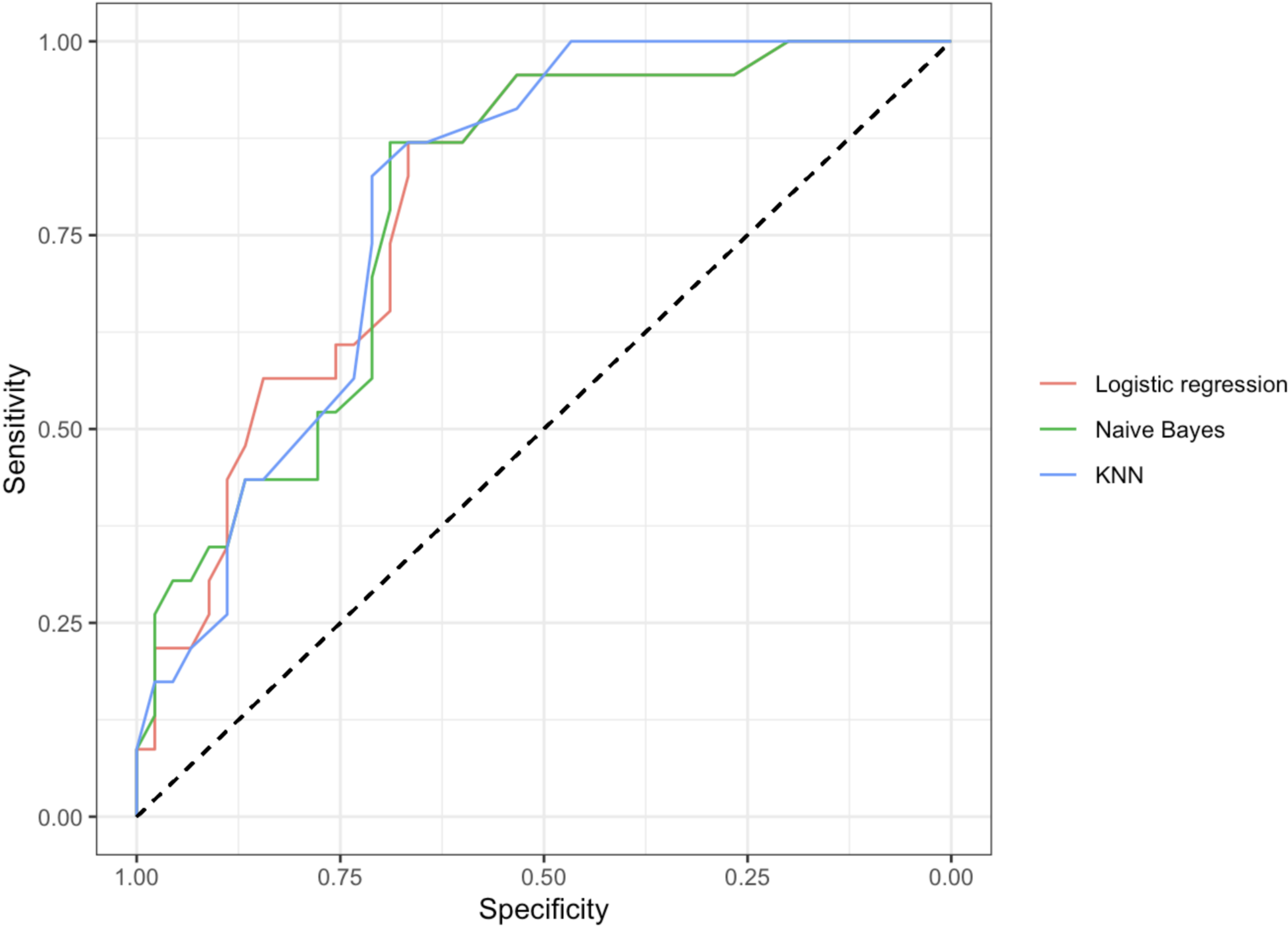
Comparing models

- The three models can be compared with ROC curves and AUROC

```
roc_logit_cv <- roc(test_set$y, p_hat_logit_cv)
roc_nb_cv    <- roc(test_set$y, p_hat_nb_cv)
roc_knn_cv   <- roc(test_set$y, p_hat_knn_cv)

ggroc(list("Logistic regression" = roc_logit_cv,
          "Naive Bayes" = roc_nb_cv,
          "KNN" = roc_knn_cv)) +
  theme(legend.title = element_blank()) +
  geom_segment(aes(x = 1, xend = 0, y = 0, yend = 1),
              color = "black",
              linetype = "dashed") +
  ylab("Sensitivity") +
  xlab("Specificity")
```

Comparing models



```
auc(roc_logit_cv)
```

```
## Area under the curve: 0.794
```

```
auc(roc_nb_cv)
```

```
## Area under the curve: 0.788
```

```
auc(roc_knn_cv)
```

```
## Area under the curve: 0.797
```

Comparing models

Algorithm	Overall accuracy	Sensitivity	Specificity	F ₁ score	AUROC
Logistic regression	0.706	0.2174	0.9556	0.333	0.794
Naïve Bayes	0.706	0.3480	0.8890	0.444	0.788
KNN	0.676	0.2609	0.8889	0.353	0.797

For a 0.5 threshold, KNN has the worst overall accuracy, but the second best sensitivity and specificity. It also has the highest AUROC, but the second best F₁ score. All of the models perform similarly and a choice in best model should consider a different threshold and whether maximizing sensitivity is preferred over maximizing specificity.